

Week 4: Feedforward Networks & Backpropagation

Deep Learning – IITM BS
Notes with Intuition + Diagrams

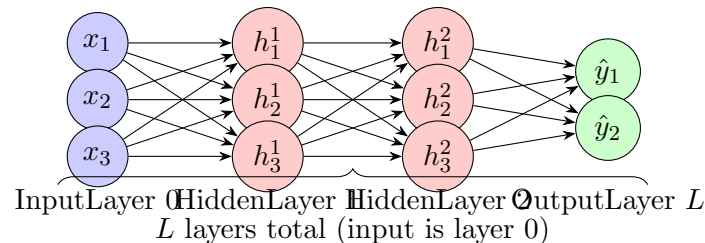
Contents

1	Feedforward Neural Network Structure	2
2	Choosing Output Activation & Loss Function	2
3	Backpropagation: The Big Idea	3
3.1	Step 1: Gradient w.r.t. Output Layer	3
3.2	Step 2: Gradient w.r.t. Hidden Units (Chain Rule with Multiple Paths)	3
3.3	Step 3: Gradient w.r.t. Parameters	4
4	Full Forward & Backward Pass Algorithms	4
5	Why Backpropagation is Efficient: Dynamic Programming View	4
6	Practice Problems	5
7	Solutions	6

1 Feedforward Neural Network Structure

Intuition:

Stacking neurons into layers A single sigmoid neuron can only draw one smooth curve boundary. Stack many of them into a **hidden layer**, then stack multiple hidden layers, and you get a network that can approximate *any* continuous function (Universal Approximation Theorem). Depth lets the network build complex features hierarchically: early layers detect simple patterns, later layers combine them into complex ones.



FFNN Equations

Pre-activation at layer i : $a_i(x) = b_i + W_i h_{i-1}(x)$

Activation at layer i : $h_i(x) = g(a_i(x))$ (g = activation function, applied element-wise)

Boundary conditions: $h_0 = x$ (input), $h_L = \hat{y} = \hat{f}(x)$ (output)

Output layer: $f(x) = h_L(x) = O(a_L(x))$ where O is the output activation.

Dimensions: $W_i \in \mathbb{R}^{n \times n}$, $b_i \in \mathbb{R}^n$ for hidden layers; $W_L \in \mathbb{R}^{k \times n}$, $b_L \in \mathbb{R}^k$ for output (k = number of classes).

Intuition:

Why separate "pre-activation" a from "activation" h ? This separation is exactly what makes backpropagation clean. a_i is the raw weighted sum (linear operation, easy to differentiate w.r.t. weights). h_i is what happens after squashing through a nonlinearity g (easy to differentiate w.r.t. a_i using g'). Splitting these lets us apply the chain rule cleanly, one simple step at a time.

2 Choosing Output Activation & Loss Function

Task	Output Activation	Loss Function
Regression (real values)	Linear	Mean Squared Error (MSE)
Classification (probabilities)	Softmax	Cross-Entropy

Loss Functions

MSE: $L(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^k (\hat{y}_{ij} - y_{ij})^2$

Cross-Entropy (for one-hot labels): $L(\theta) = -\frac{1}{N} \sum_i \sum_j y_{ij} \log(\hat{y}_{ij})$

Softmax: $\hat{y}_j = \frac{e^{a_{L,j}}}{\sum_i e^{a_{L,i}}}$ (converts raw scores into a probability distribution)

Intuition:

Why cross-entropy pairs so naturally with softmax. Cross-entropy heavily penalizes *confident but wrong* predictions (as $\hat{y} \rightarrow 0$ for the true class, $-\log \hat{y} \rightarrow \infty$). This gives a strong gradient signal exactly when the model is most wrong. Softmax ensures outputs form a valid probability distribution (sum to 1, all positive), which cross-entropy needs to make sense as “surprise” or “information loss”.

3 Backpropagation: The Big Idea

Intuition:

”Talk to” each part of the network. Backpropagation is fundamentally the **chain rule**, applied systematically. Imagine you want to know: “How much does changing this one weight, buried deep in the network, affect the final loss?” You can’t compute this directly—but you *can* pass a message backwards: first ask “how does the loss change with the output?”, then “how does the output change with the last hidden layer?”, and so on, layer by layer, back to the weight you care about. Each step is a simple local derivative; the chain rule links them all together.

$$\frac{\partial \mathcal{L}(\theta)}{\partial W_{111}} = \underbrace{\frac{\partial \mathcal{L}(\theta)}{\partial \hat{y}}}_{\text{Talk to output}} \underbrace{\frac{\partial \hat{y}}{\partial a_3} \frac{\partial a_3}{\partial h_2}}_{\text{Talk to hidden layers}} \underbrace{\frac{\partial h_2}{\partial a_2} \frac{\partial a_2}{\partial h_1} \frac{\partial h_1}{\partial a_1}}_{\text{Talk to weight}} \frac{\partial a_1}{\partial W_{111}}$$

3.1 Step 1: Gradient w.r.t. Output Layer

For softmax + cross-entropy (the most common combo), there’s a beautiful simplification:

Softmax + Cross-Entropy Gradient Shortcut – MEMORIZE

$$\frac{\partial \mathcal{L}(\theta)}{\partial a_{L,i}} = \hat{y}_i - y_i \quad (\text{or in vector form: } \hat{\mathbf{y}} - \mathbf{e}_l)$$

where $\mathbf{e}_l$ is the one-hot vector for the true class l .

Intuition:

Why does this simplify so beautifully? Naively, you’d expect a messy derivative because $\hat{y}_j$ depends on *all* the $a_{L,i}$ (through the softmax normalization). But when you carefully differentiate $-\log \hat{y}_l$ with respect to each $a_{L,i}$ using the quotient rule for softmax, all the messy exponential terms **cancel out**, leaving just “predicted minus true.” This is the single most useful shortcut in deep learning practice—it means the error signal starting backpropagation is simply the model’s mistake vector!

3.2 Step 2: Gradient w.r.t. Hidden Units (Chain Rule with Multiple Paths)

Intuition:

Why do multiple paths matter here? A single hidden neuron in layer i doesn’t just feed into *one* neuron in the next layer—it feeds into **all** of them (since layers are fully connected). So when computing “how does changing h_{ij} affect the loss”, we must **sum the contributions coming back through every downstream neuron** it influences. This is the multivariable chain rule: $\frac{\partial p(z)}{\partial z} = \sum_m \frac{\partial p(z)}{\partial q_m} \frac{\partial q_m}{\partial z}$.

Gradient w.r.t. Hidden Layers

$$\frac{\partial \mathcal{L}(\theta)}{\partial h_{ij}} = \sum_{m=1}^k \frac{\partial \mathcal{L}(\theta)}{\partial a_{i+1,m}} W_{i+1,m,j}$$

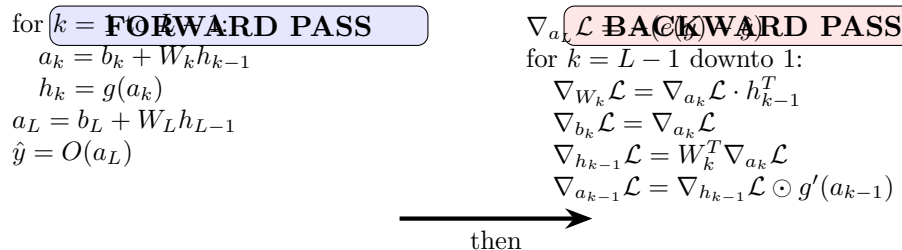
$$\frac{\partial \mathcal{L}(\theta)}{\partial a_{ij}} = \frac{\partial \mathcal{L}(\theta)}{\partial h_{ij}} \cdot g'(a_{ij})$$

3.3 Step 3: Gradient w.r.t. Parameters**Gradient w.r.t. Weights and Biases**

$$\frac{\partial \mathcal{L}(\theta)}{\partial W_{kij}} = \frac{\partial \mathcal{L}(\theta)}{\partial a_{ki}} \cdot h_{k-1,j}^T \quad \frac{\partial \mathcal{L}(\theta)}{\partial b_{ki}} = \frac{\partial \mathcal{L}(\theta)}{\partial a_{ki}}$$

Intuition:

Weight gradient = "error at destination $\times$ activation at source" This pattern is universal across all of deep learning (it's exactly what you saw for the single sigmoid neuron in Week 3!). The gradient for a weight connecting neuron j (previous layer) to neuron i (current layer) is simply: how wrong neuron i 's pre-activation is ($\frac{\partial \mathcal{L}}{\partial a_{ki}}$), multiplied by how active neuron j was ($h_{k-1,j}$). If the source neuron never fired ($h = 0$), that weight gets zero blame—makes intuitive sense, since it didn't contribute anything!

4 Full Forward & Backward Pass Algorithms**Activation Derivatives (used as g')**

Sigmoid: $g'(z) = g(z)(1 - g(z))$

tanh: $g'(z) = 1 - g(z)^2$

ReLU: $g'(z) = 1$ if $z > 0$, else 0

5 Why Backpropagation is Efficient: Dynamic Programming View**Intuition:**

Reusing computation Notice how $\nabla_{a_{k-1}} \mathcal{L}$ depends on $\nabla_{a_k} \mathcal{L}$, which depends on $\nabla_{a_{k+1}} \mathcal{L}$, and so on. If we computed gradients "from scratch" for every weight independently, we'd redo the same work millions of times. Instead, backprop computes each layer's gradient signal **once** and reuses it for every weight in that layer—exactly like memoization in dynamic programming. This is why training networks with millions of parameters is computationally feasible at all.

6 Practice Problems

Problem 1: Forward Pass [3 marks]

2-layer network. $x = [1, 2]^T$. $W_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, $b_1 = [0, 0]^T$, sigmoid hidden activation. $W_2 = [1, 1]$, $b_2 = 0$, linear output.
Compute h_1 (hidden activation) and $\hat{y}$.

Problem 2: Softmax Output [2 marks]

Pre-softmax scores: $a_L = [2, 1, 0]$. Compute the softmax output $\hat{y}$ (round to 3 decimals).

Problem 3: Cross-Entropy Gradient [2 marks]

Using $\hat{y}$ from Problem 2, if the true class is class 0 (one-hot $[1, 0, 0]$), compute $\nabla_{a_L} \mathcal{L} = \hat{y} - y$.

Problem 4: Backprop Through One Hidden Layer [3 marks]

Given $\nabla_{a_2} \mathcal{L} = [0.3, -0.2]^T$ (gradient at output pre-activation), $W_2 = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix}$, h_1 pre-activation values $a_1 = [0.5, -0.5]^T$, sigmoid hidden activation.
Compute $\nabla_{h_1} \mathcal{L}$ and then $\nabla_{a_1} \mathcal{L}$.
(Hint: $\sigma(0.5) \approx 0.622$, $\sigma(-0.5) \approx 0.378$.)

Problem 5: Weight Gradient [2 marks]

$\nabla_{a_k} \mathcal{L} = [0.4, -0.1]^T$, $h_{k-1} = [1, 2, 0]^T$. Compute $\nabla_{W_k} \mathcal{L}$ (should be 2×3 matrix).

Problem 6: MCQ – Activation Derivative [2 marks]

For a tanh-activated hidden neuron with $a = 0$, what is $g'(a)$?
A. 0 B. 0.5 C. 1 D. 0.25

7 Solutions

Problem 1: $a_1 = W_1x + b_1 = [1, 2]^T$. $h_1 = \sigma([1, 2]) = [0.731, 0.881]$.

$$\hat{y} = W_2h_1 + b_2 = 1(0.731) + 1(0.881) = \mathbf{1.612}.$$

Problem 2: $e^2 = 7.389, e^1 = 2.718, e^0 = 1$. Sum = 11.107.

$$\hat{y} = [7.389/11.107, 2.718/11.107, 1/11.107] = [\mathbf{0.665}, \mathbf{0.245}, \mathbf{0.090}].$$

Problem 3: $\nabla_{a_L}\mathcal{L} = \hat{y} - y = [0.665 - 1, 0.245 - 0, 0.090 - 0] = [-\mathbf{0.335}, \mathbf{0.245}, \mathbf{0.090}]$.

Problem 4: $\nabla_{h_1}\mathcal{L} = W_2^T \nabla_{a_2}\mathcal{L} = \begin{bmatrix} 1 & 0 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 0.3 \\ -0.2 \end{bmatrix} = \begin{bmatrix} 0.3 \\ -0.3 - 0.4 \end{bmatrix} = \begin{bmatrix} 0.3 \\ -0.7 \end{bmatrix}$.

$g'(a_1) = \sigma(a_1)(1 - \sigma(a_1))$: at $a = 0.5$: $0.622(0.378) = 0.235$; at $a = -0.5$: $0.378(0.622) = 0.235$.

$$\nabla_{a_1}\mathcal{L} = [0.3, -0.7] \odot [0.235, 0.235] = [\mathbf{0.0705}, \mathbf{-0.1645}].$$

Problem 5: $\nabla_{W_k}\mathcal{L} = \nabla_{a_k}\mathcal{L} \cdot h_{k-1}^T = \begin{bmatrix} 0.4 \\ -0.1 \end{bmatrix} [1, 2, 0] = \begin{bmatrix} 0.4 & 0.8 & 0 \\ -0.1 & -0.2 & 0 \end{bmatrix}$.

Problem 6: C. $\tanh'(0) = 1 - \tanh^2(0) = 1 - 0 = 1$.